

You're almost there — sign up to start building in Notion today.

Sign up or login

Spec Document

1. Overview

Replace the stub in `database/db.py` with a working SQLite implementation.

This step establishes the **data layer foundation** for the Spendly application.

All future features (authentication, profile, expense tracking) depend on this being correctly implemented.

2. Depends on

Nothing — this is the first step.

3. Routes

- No new routes
 - Existing placeholder routes in `app.py` remain unchanged
-

4. Database Schema

A. users

Column	Type	Constraints
id	INTEGER	Primary key, autoincrement
name	TEXT	Not null
email	TEXT	Unique, not null
password_hash	TEXT	Not null
created_at	TEXT	Default datetime('now')

B. expenses

Column	Type	Constraints
id	INTEGER	Primary key, autoincrement
user_id	INTEGER	Foreign key → users.id, not null
amount	REAL	Not null
category	TEXT	Not null
date	TEXT	Not null (YYYY-MM-DD format)
description	TEXT	Nullable
created_at	TEXT	Default datetime('now')

5. Functions to Implement (`database/db.py`)

A. `get_db()`

- Opens connection to `spendly.db` (or `expense_tracker.db`) in project root

- Sets:
 - `row_factory = sqlite3.Row`
 - `PRAGMA foreign_keys = ON`
 - Returns the connection
-

B. `init_db()`

- Creates both tables using `CREATE TABLE IF NOT EXISTS`
 - Safe to call multiple times
 - Ensures schema is ready before app usage
-

C. `seed_db()`

- Checks if `users` table already contains data
 - If yes → return early (no duplication)
 - Inserts one demo user:
 - name: Demo User
 - email: `demo@spendly.com`
 - password: demo123 (hashed using `werkzeug`)
 - Inserts **8 sample expenses**:
 - All linked to demo user
 - Cover multiple categories
 - Dates spread across current month
 - At least one expense per category
-

6. Changes to `app.py`

- Import:
 - `get_db`
 - `init_db`
 - `seed_db`
 - Call `init_db()` and `seed_db()` inside `app.app_context()` on startup
 - Ensure DB is ready before routes are used
-

7. Files to Change

- `database/db.py` → implement all functions
 - `app.py` → add imports and startup calls
-

8. Files to Create

- None
-

9. Dependencies

- No new pip packages
 - Use:
 - `sqlite3` (standard library)
 - `werkzeug.security` (already installed)
-

10. Categories (Fixed List)

Use exactly these values:

- Food
- Transport
- Bills

- Health
 - Entertainment
 - Shopping
 - Other
-

11. Rules for Implementation

- No ORMs (no SQLAlchemy)
- Use **parameterized queries only**
- Never use string formatting in SQL
- Enable `PRAGMA foreign_keys = ON` on every connection
- Store `amount` as REAL (float), not INTEGER
- Hash passwords using:

```
fromwerkzeug.securityimportgenerate_password_hash
```

- `seed_db()` must prevent duplicate inserts
 - Dates must follow **YYYY-MM-DD format consistently**
-

12. Expected Behavior

- `get_db()` returns a working connection with:
 - dictionary-like row access
 - foreign key enforcement enabled
- `init_db()`:
 - creates tables safely
 - does not fail on repeated runs

- `seed_db()` :
 - inserts demo data only once
 - does not duplicate records on multiple runs
 - Database enforces:
 - unique email constraint
 - valid foreign key relationships
-

13. Error Handling Expectations

- Inserting duplicate email → should fail (UNIQUE constraint)
 - Inserting expense with invalid `user_id` → should fail (foreign key constraint)
 - Invalid queries → should raise clear errors for debugging
-

14. Definition of Done

- ☐ Database file is created on app startup
- ☐ Both tables exist with correct schema and constraints
- ☐ Demo user exists with hashed password